Entrega Trabajo en clase: API REST (Avistamientos)

Este es el link del repositorio: https://github.com/dsuarezga-pro/Software2Repository

Justificación de Pruebas

1. Cambios en el código

Se extrajo la lógica de validación del controlador y se encapsuló en una nueva clase ValidadorDeAvistamiento. Adicionalmente, se creó una interfaz Reloj (con su implementación RelojSistema) para manejar la obtención de la fecha actual. Esto hace que la API sea más testeable porque permite probar la lógica de validación de forma aislada sin necesidad de levantar el servidor web ni depender de la fecha real del sistema (usando un stub del reloj), garantizando pruebas deterministas.

Este es el link del repositorio:

https://github.com/dsuarezga-pro/Software2Repository

2. Tabla de casos de prueba (Parte A)

Campo Evaluado	Valor de Prueba	Tipo de Caso	Resultado Esperado
Especie	"Pi" (2 caracteres)	Valor Límite (Inválido)	Excepción: 400 Bad Request
Especie	"Pez" (3 caracteres)	Valor Límite (Válido)	Pasa validación
Especie	"Colibrí" (7 caracteres)	Representativo (Válido)	Pasa validación
Especie	Texto de 50 caracteres	Valor Límite (Válido)	Pasa validación
Especie	Texto de 51 caracteres	Valor Límite (Inválido)	Excepción: 400 Bad Request
Fecha	Fecha actual (Hoy)	Valor Límite (Válido)	Pasa validación
Fecha	Fecha de mañana	Valor Límite (Inválido)	Excepción: 400 Bad Request
Lugar / Observador	"" (Cadena vacía)	Partición (Inválido)	Excepción: 400 Bad Request

Justificación:

Se aplicó partición de equivalencia y valores límite. Para la especie (3-50 caracteres), probar "Pi" (2) y un texto de 51 caracteres verifica los límites inválidos, mientras que "Pez" (3) y un texto de 50 verifican los límites válidos.

3. Por qué cada nivel

- Unitaria (ValidadorDeAvistamientoTest): Se usó para probar los límites de los campos (especie de 2 o 51 caracteres). Esto es unitario porque prueba la lógica de negocio pura en aislamiento, ejecutándose en milisegundos sin levantar Spring Boot.
- Integración (AvistamientoControllerTest): Se usó para probar el flujo completo (verificar que un POST exitoso retorna 201 y un DELETE retorna 204 y afecta posteriores GETs).

Esto detecta errores de configuración de Spring, de serialización JSON y de comunicación con la base de datos (H2) que las pruebas unitarias del validador jamás detectarían.

Para ilustrar de forma práctica la diferencia entre ambos niveles de prueba implementados en el proyecto, se detallan a continuación casos específicos:

Pruebas Unitarias (Ej. ValidadorDeAvistamientoTest)

- Prueba de límite inválido (Especie corta): Evalúa la regla de negocio pura cuando se ingresa una especie de 2 caracteres ("Pi"). Verifica que se lance una excepción IllegalArgumentException sin necesidad de levantar Spring Boot ni interactuar con la base de datos.

```
@Test
void validar_EspecieDe2Caracteres_LanzaExcepcion() {
    dto.setEspecie("Pi");
    IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> {
        validador.validar(dto);
    });
    assertTrue(exception.getMessage().contains("entre 3 y 50 caracteres"));
}
```

- Prueba con Stub del Reloj (Fecha del futuro): Utiliza un objeto simulado (Stub) de la interfaz Reloj para congelar la fecha actual ("2026-10-21") y evaluar que una fecha posterior (mañana) sea rechazada de forma determinista, independientemente de la fecha real del sistema.

```
@Test
void validar_FechaDeManana_LanzaExcepcion() {
    LocalDate manana = HOY.plusDays(1);
    dto.setFecha(manana);
    IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> {
        validador.validar(dto);
    });
    assertTrue(exception.getMessage().contains("posterior a hoy"));
}
```

Pruebas de Integración (Ej. AvistamientoControllerTest)

- Prueba de creación exitosa (POST -> 201 Created): Simula una petición HTTP real mediante MockMvc para enviar un JSON completo. Verifica que el sistema responda con código HTTP 201, que la capa JPA persista el registro en la base de datos en memoria H2 y que se devuelva un ID autogenerado.

```java
// 1. POST - Éxito (201) y verificación de respuesta completa
@Test
void deberiaCrearAvistamientoYRetornar201() throws Exception {
    String json = """
            {
                "especie": "Colibrí",
                "lugar": "Jardín Botánico",
                "fecha": "2026-10-10",
                "observador": "Juan Pérez"
            }
            """;

    mockMvc.perform(post("/avistamientos").contentType(MediaType.APPLICATION_JSON).content(json)).andDo(print())
            .andExpect(status().isCreated()).andExpect(jsonPath("$.id").exists())
            .andExpect(jsonPath("$.especie").value("Colibrí"))
            .andExpect(jsonPath("$.lugar").value("Jardín Botánico"));
}
```

- Prueba de efecto secundario (DELETE -> 204 y su impacto en GET -> 404): Demuestra la integración completa del ciclo de vida de los datos al eliminar un recurso existente (esperando un código 204 No Content) y realizar inmediatamente una consulta GET posterior para comprobar que el sistema devuelve un error 404 Not Found, validando la persistencia real.

```java
// 6. DELETE - Éxito (204) y verificación del efecto
@Test
void deberiaEliminarAvistamientoYComprobarEfecto404() throws Exception {
    String jsonCrear = "{\"especie\": \"Pato\", \"lugar\": \"Lago\", \"fecha\": \"2026-01-01\", \"observador\": \"Luis\"}";

    org.springframework.test.web.servlet.MvcResult result = mockMvc
            .perform(post("/avistamientos").contentType(MediaType.APPLICATION_JSON).content(jsonCrear)).andReturn();
    Integer idDinámico = com.jayway.jsonpath.JsonPath.read(result.getResponse().getContentAsString(), "$.id");

    mockMvc.perform(delete("/avistamientos/" + idDinámico)).andDo(print()).andExpect(status().isNoContent());

    mockMvc.perform(get("/avistamientos/" + idDinámico)).andDo(print()).andExpect(status().isNotFound());
}
```

4. Los dobles

Se utilizó un Stub (doble de prueba) de la interfaz Reloj usando Mockito (mock(Reloj.class) y when(relojStub.hoy()).thenReturn(...)). Se eligió un stub porque solo necesitábamos forzar una respuesta indirecta (una fecha congelada "2026-10-21") para alimentar al sistema bajo prueba. Sin este doble, la prueba de "fecha de mañana" habría pasado el día en que se escribió, pero fallaría un mes después.

5. Lo que no probamos

- No se escribió una prueba unitaria para los métodos getter/setter de la clase AvistamientoDTO, ya que son generados automáticamente o muy simples (no contienen lógica que pueda fallar por sí sola).
- No se probaron fallos de conexión de red ni caídas de la base de datos en las pruebas de integración, ya que el enfoque del taller es la lógica de negocio y el comportamiento esperado de la API, no la resiliencia de la infraestructura.

6. Lectura de la cobertura

Al ejecutar el reporte con JaCoCo, se obtuvo una cobertura total de instrucciones del 80% para todo el proyecto. Al analizar el reporte, se observa lo siguiente:

api-avistamientos

api-avistamientos

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.aves.api.service		74%		50%	4	13	5	32	3	12	0	1
com.aves.api.dto		62%		n/a	5	16	10	27	5	16	1	2
com.aves.api.controller		73%		n/a	3	8	3	14	3	8	0	1
com.aves.api.util		93%		77%	4	13	1	16	0	4	0	2
com.aves.api		37%		n/a	1	2	2	3	1	2	0	1
com.aves.api.model		100%		n/a	0	12	0	24	0	12	0	1
com.aves.api.exception		100%		n/a	0	5	0	9	0	5	0	2
Total	92 of 478	80%	5 of 20	75%	17	69	21	125	12	59	1	10

- El paquete com.aves.api.dto tiene una cobertura del 62%. Esto indica que existen ramas y líneas sin cubrir correspondientes a métodos generados automáticamente (getters, setters y constructores por defecto).
- El paquete com.aves.api.util, que contiene la lógica de negocio extraída, alcanzó una cobertura del 93%, demostrando la efectividad de las pruebas unitarias aisladas.
- Conclusión: Se decidió conscientemente no escribir pruebas para cubrir estas líneas faltantes en los DTOs. Escribir pruebas unitarias dedicadas exclusivamente a invocar un get o un set no aporta ningún valor a la verificación del comportamiento del sistema y solo sirve para inflar artificialmente la métrica. Tal como se abordó en clase, una suite de pruebas modesta (80%) que verifica el comportamiento real de la API y las reglas de negocio tiene mucho más valor que un 100% lleno de pruebas sin aserciones útiles.